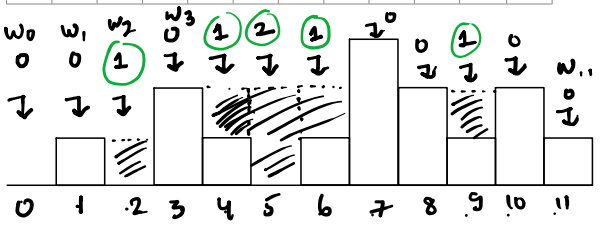
Rain-Water Trapping

Given an array of **N** integers representing height of **N** buildings, find the **total** amount of **rainwater** that can be trapped between the building such width of each building is **1 unit**.

Example :

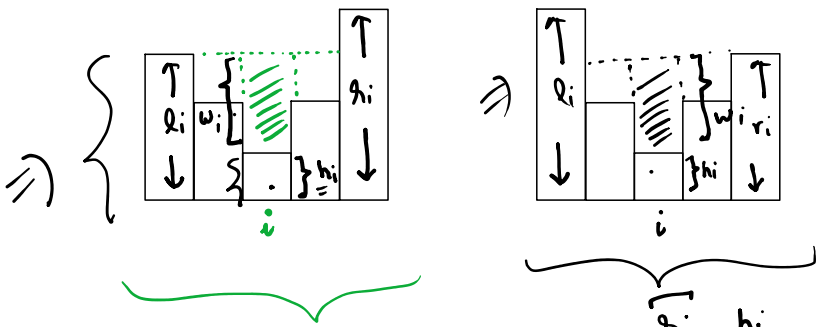
0	1	2	3	4	5	6	7	8	9	10	11
0	1	0	2	1	0	1	3	2	1	2	1



o/p : 6

$$\text{total water} = \sum_{i=0}^{n-1} [w_i]$$

water trapped on top of i^{th} building



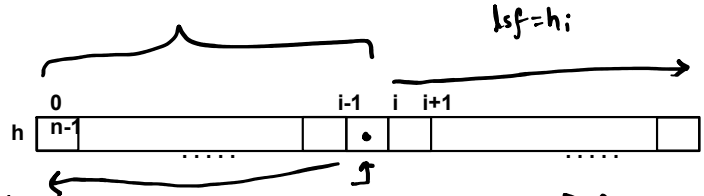
$w_i = l_i - h_i$

$w_i = r_i - h_i$

$w_i = \min(l_i, r_i) - h_i$

$l_i = (0 \text{ to } i)\ max$ $\Rightarrow r_i = (i \text{ to } n-1)\ max$

$w_i = 0$



$lsf = h_i$

$\Rightarrow l_i = \max(h[0 \ldots i])$

$(h[i \ldots n-1])$

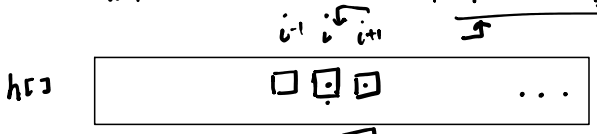
lsf = hi

$l_i = \max(h[0 \ldots i])$

$w_i = \min(l_i, r_i) - h_i$ $\quad r_i = \max(h[i \ldots n-1])$

$r_{n-1} = h_{n-1}$

$r_i = \max(r_{i+1}, h_i)$



$r_{i+1} = \max(h[i+1 \ldots n-1])$

$= \max(r_{i+1}, h_i)$

$r_i = \max(h[i \ldots n-1])$

$w_i = \min(l_i, r_i) - h[i]$

$l_i = \max(h[0 \ldots i]) = \max(l_{i-1}, h_i)$

$i \geq 1$

$\Rightarrow l_{i-1} = \max(h[0 \ldots i-1])$

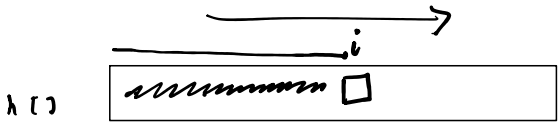
$l_0 = h_0$

$w_7 =$

0	0	1	2	3	4	5	6	7	8	9	10	11
h	0	1	0	2	1	0	1	3	2	1	2	1
l	0	1	1	2	2	2	2	3	3	3	3	3
r	3	3	3	3	3	3	3	3	2	2	2	1

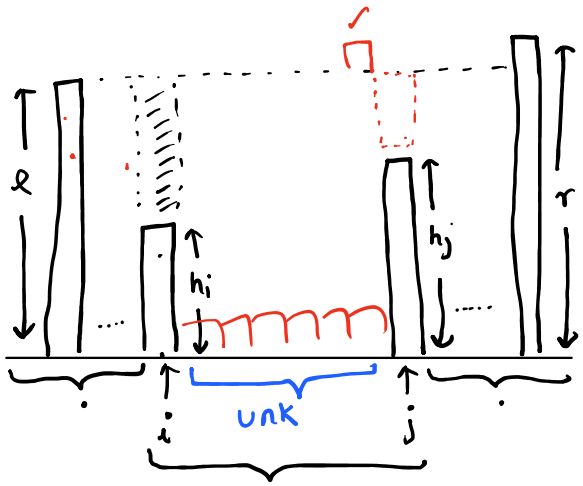
$l_i = \max(l_{i-1}, h_i)$

$r_i = \max(r_{i+1}, h_i)$



$w_i = \min(l_i, r_i) - h[i]$

$\max(h[0 \ldots i])$ $\quad \max(h[i \ldots n-1])$



if ($l < r$) {

$\Rightarrow w_i = l - h_i$

i++;

$l = \max(h[0 \ldots i])$

$r = \max(h[j \ldots n-1])$

$i \leq j$

} else {

$w_j = l - h_j$

j--;

}

Product of Array Except Self

Given an integer array **nums**, return *an array* **answer** *such that* **answer[i]** *is equal to the product of all the elements of* nums **except** nums[i].

The product of any prefix or suffix of nums is **guaranteed** to fit in a **32-bit** integer.

Example :

Input

0	1	2	3	4
1	2	3	4	5

Output

0	1	2	3	4
120	60	40	30	24

[R][HW] 3Sum

[HW] https://leetcode.com/problems/3sum-closest/
[HW] https://leetcode.com/problems/4sum/

3Sum

Given an integer array **nums**, return all the triplets { $nums_i$, $nums_j$, $nums_k$ } such that i != j, i != k and j != k and $nums_i$ + $nums_j$ + $nums_k$ == 0.

Notice that the solution set must not contain duplicate triplets.

Example

Input

0	1	2	3	4	5
-1	0	1	2	-1	-4

[HW] 3Sum Closest

[HW] https://leetcode.com/problems/3sum-closest/

[HW] 4Sum

[HW] https://leetcode.com/problems/4sum/

DNF (Dutch National Flag) Sort

time: O(n)

Given an array containing **0s**, **1s** & **2s** design an algorithm to **sort** the array.

Example

Input

N = 9

0	1	2	3	4	5	6	7	8
1	0	1	2	0	1	2	0	1

Output

0	1	2	3	4	5	6	7	8
0	0	0	1	1	1	1	2	2

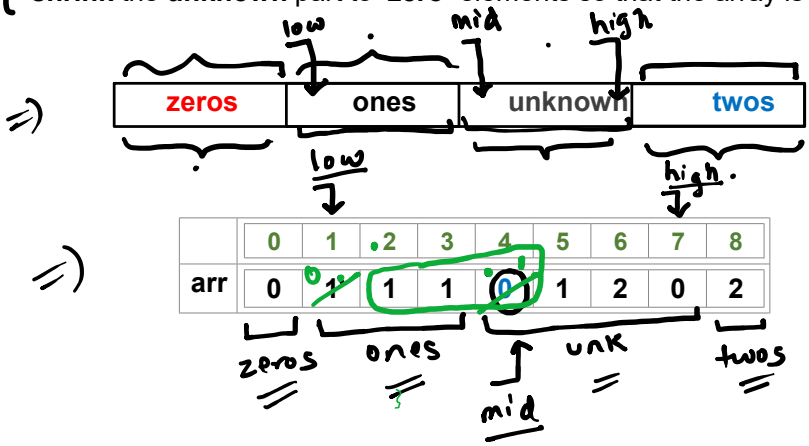
[0...low-1] ⇒ zeros
[low...mid-1] ⇒ ones
[mid...high] ⇒ unk
[high+1...n-1] ⇒ twos

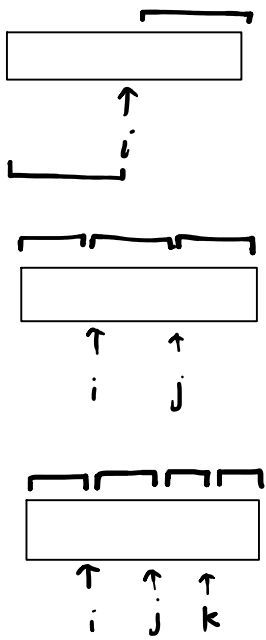
mid ⇒ points to the 1st el. of unk region

low ⇒ points to the loc. where you'll put the next zero

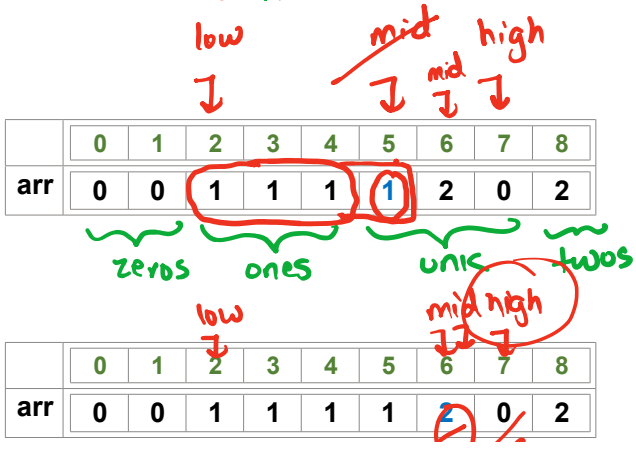
high ⇒ points to the loc. where you'll put the next two

The idea behind the **DNF** algorithm is to **partition** the given array into four parts using three pointers - *low*, *mid* & *high* and then to iteratively **shrink** the **unknown** part to **zero** elements so that the array is sorted.



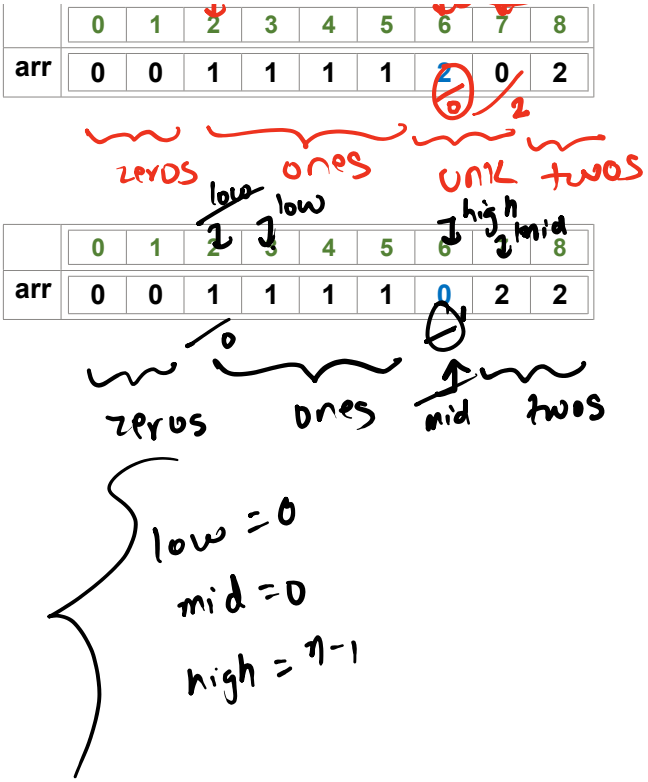


if (arr(mid) == 0) { swap (arr[mid], arr[low]) low++ mid++ }



if (arr(mid) == 1) mid++

swap(arr(mid), arr(high)) high--;

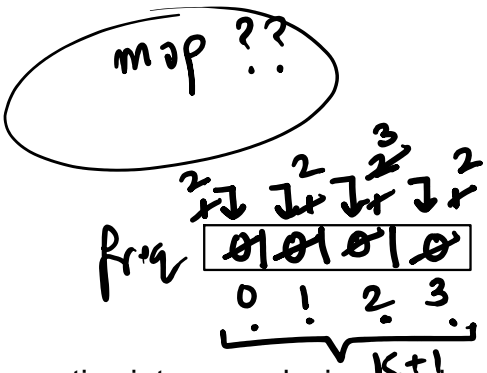


high--;

low = 0

mid = 0

high = n-1



Counting Sort

Given an array of **N** non-negative integers, design an algorithm to **sort** the array such that each array element is **<=** a given integer **K**.

el ∈ [0,K]

Example

Input

N = 9, K = 3

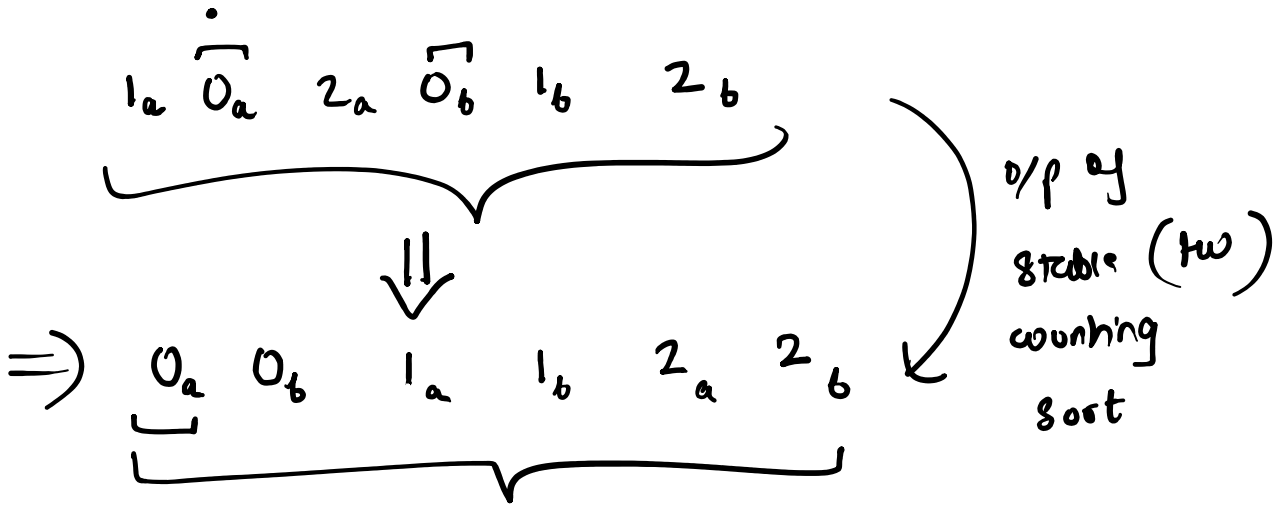
0	1	2	3	4	5	6	7	8
1	0	3	2	3	1	2	0	2

el. freq
0 → 2
1 → 2
2 → 3
3 → 2

0 0 1 1 2 2 2 3 3

Output

0	1	2	3	4	5	6	7	8
0	0	1	1	2	2	2	3	3



Generalized Counting Sort

Given an array of **n** integers, design an algorithm to **sort** the array such that each array element is **in the range [l, r]** , assume l <= r

Example

Input

n = 10, l = 2, r = 4

0	1	2	3	4	5	6	7	8	9
4	3	2	2	4	3	5	4	5	2

Output

0	1	2	3	4	5	6	7	8	9
2	2	2	3	3	4	4	4	5	5